



AI FULL-STACK & DEPLOYMENT ENGINEER · TRACK 2.A · SESSION 5

Safe Model Upgrades & Incident Response

Shadow deployments that expose zero users, trace-level comparison of model versions, evidence-based go/no-go decisions — and a disciplined response when production fails anyway

Traffic mirroring · Langfuse tracing · v1 vs v2 comparison · Go/no-go framework · Severity levels · Runbooks · Blameless postmortems

The lab runs entirely on your local machine; a free Langfuse Cloud account is a supported optional path.

Day 5 · 2 hours · concepts 45 min + local lab 60 min + review 15 min



SESSION 5 · AGENDA

Session agenda

45 min



Concepts & walkthrough

Why a model upgrade is a release of probabilistic behaviour; upgrade triggers; shadow versus canary; traffic mirroring; Langfuse tracing and version comparison; the go/no-go framework; severity levels, runbooks, rollback-first discipline and blameless postmortems.

60 min



Local shadow-deployment lab

A traffic mirror duplicates every request to a v2 candidate at zero user exposure; both versions emit Langfuse traces tagged by version. Compare, decide go/no-go on evidence — then an incident drill: detect a broken candidate, execute the rollback runbook, write the postmortem.

15 min



Review & wrap-up

Compare go/no-go decisions and postmortems across pairs, map each local mechanism to its production counterpart, and preview Track 2.B.

Objective: validate a model upgrade on full production traffic without exposing a single user, decide on evidence — and respond to an incident with a rehearsed, documented procedure.



THE PROBLEM

An upgrade is a release of probabilistic behaviour

The upgrade problem

New weights or prompt policy = new behaviour everywhere

- A model upgrade shifts the probability distribution of every answer — not one code path, all of them
- Health checks, unit tests and the API contract all stay green while quality, tone, latency and cost move
- Offline evaluations sample expected questions; production supplies the phrasing and edge cases the suite has never seen
- Betting production on an untested upgrade is therefore unacceptable — the candidate must be observed under real traffic first

The incident problem

Failures still happen — the response must be disciplined

- Gates and progressive delivery reduce incident frequency; they do not reduce it to zero
- An unrehearsed response improvises under pressure: diagnosis before mitigation, undocumented commands, blame afterwards
- A disciplined response is classified by severity, guided by a runbook, mitigated by rollback first, and closed by a blameless postmortem
- Both halves are engineering artifacts: the mirror and the runbook are written, versioned and rehearsed before they are needed

Principle: upgrades earn production traffic through observed evidence, and incidents are met with rehearsed procedure. This session builds both: the shadow pipeline and the response discipline.



CORE CONCEPTS

What triggers a model upgrade or retraining



Drift breach (Session 4)

The drift monitors fire: input distributions have shifted beyond threshold. The world changed; the deployed behaviour was tuned for the old world.



Scheduled refresh

A standing cadence — retrain or re-prompt quarterly, refresh embeddings monthly. Model currency treated as routine, planned maintenance.



Live-eval drop

Evaluation scores computed on sampled production traffic decline week over week, even without visible input drift. Quality is eroding in place.



New base model

The provider ships a better or cheaper model — or deprecates the current one. Vendor-driven, sometimes with a hard migration deadline.

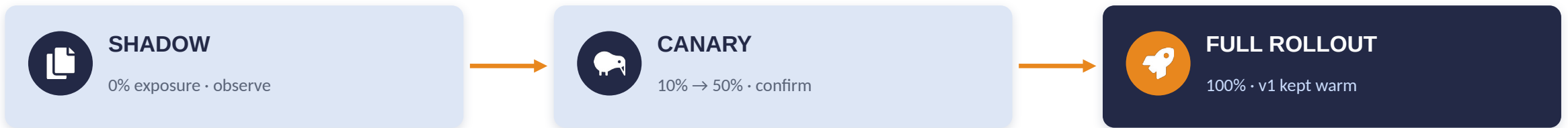
Common path: whatever the trigger, the candidate enters the same pipeline — offline evals (Session 3), then shadow under real traffic, then canary, then full rollout. No trigger is urgent enough to skip the evidence.



CORE CONCEPTS

Shadow versus canary: exposure against feedback

	Shadow (today)	Canary (Session 3)
User exposure	0% — users always receive v1's answer; the candidate's output is never returned	Real exposure — the candidate serves 5–10% of live users from the first stage
Traffic signal	Full production distribution, duplicated request by request	Only the routed share; signal grows with each promotion stage
Feedback obtained	System evidence only: latency, errors, cost, output quality read from traces	Real user feedback and behaviour — the evidence shadow cannot produce
Cost & risk	Double inference on mirrored traffic (a real bill); zero user risk	Marginal extra capacity; bounded but real user risk at each stage



The safe-upgrade ladder: each rung is climbed only on the evidence of the previous one — shadow to observe, canary to confirm, full rollout last.



Inside the mirror: forward, duplicate, fire-and-forget

```
# mirror/mirror.py – the shadow path (abbreviated)
client = httpx.AsyncClient(timeout=Timeout(180.0))

async def shadow_to_v2(body, headers):
    try:
        r = await client.post(V2_URL, json=body,
                              headers=headers)
        log.info("SHADOW -> v2 status=%s", r.status_code)
    except Exception as exc: # NEVER reaches a user
        log.warning("SHADOW FAILED (users unaffected)")

@app.post("/generate")
async def generate(request: Request):
    body = await request.json()

    # fire-and-forget – NOT awaited on the response path
    asyncio.create_task(shadow_to_v2(body, headers))

    # the primary call: v1's answer is what users get
    r = await client.post(V1_URL, json=body,
                          headers=headers)
    return JSONResponse(status_code=r.status_code,
                        content=r.json())
```

Design properties

- The primary path never awaits the shadow call — v2's latency is invisible to users
- The shadow result is only logged; it is never returned to a caller
- Exceptions in the shadow path are caught and logged — a crashed candidate affects nobody
- The same body and auth headers are duplicated, so v2 sees exactly the production distribution

Production equivalent

Service meshes mirror below the application layer: Istio VirtualService mirror, Envoy request_mirror_policies, NGINX mirror. Identical semantics — duplicate, forward, discard the shadow response.



Langfuse: traces, observations, generations, tags, scores



Trace

One request end-to-end: input, output, duration, metadata. The unit the UI lists and filters.



Observation

A step within a trace — spans and events. Traces nest them to show structure.



Generation

The observation type for a model call: model name, input, output, token usage — the LLM-specific record.



Tags

Filterable labels on traces. The lab tags `model_version:v1 / v2` — comparison becomes two filtered views.



Scores

Quality judgments attached to traces — human review or programmatic evaluators. The hook for automated analysis.

Local path — self-hosted (today)

`docker-compose.langfuse.yml` runs Postgres 15 + `langfuse/langfuse:2` on port 3005. Trace data never leaves the machine — the self-host option many regulated clients require.

```
LANGFUSE_HOST=http://host.docker.internal:3005
```

Cloud path — optional

A free `cloud.langfuse.com` account replaces both containers. The SDK is identical — only the three environment values change. LangSmith and Arize Phoenix occupy the same category.

```
LANGFUSE_HOST=https://cloud.langfuse.com
```

Why tracing here: a shadow produces no user feedback — traces are the only evidence about the candidate. The richer the trace, the stronger the go/no-go decision.



Instrumenting the model server — tracing as an optional add-on

```
# app.py — Langfuse wired in, but never load-bearing
langfuse = None
try:
    if os.environ.get("LANGFUSE_PUBLIC_KEY") and \
        os.environ.get("LANGFUSE_SECRET_KEY"):
        from langfuse import Langfuse
        langfuse = Langfuse() # keys + host from env
except Exception:             # SDK/host problems
    langfuse = None           # -> tracing is a no-op

# inside /generate, after the model call:
if langfuse is not None:
    try:
        trace = langfuse.trace(
            name="generate", input=req.text,
            output=answer,
            tags=[f"model_version:{VERSION}"],
            metadata={"latency_ms": latency_ms})
        trace.generation(
            name="qwen-generate", model=MODEL_NAME,
            input=PROMPT_PREFIX + req.text,
            output=answer)
    except Exception as exc:
        print(f"trace emit failed (non-fatal)")
```

Two disciplines in this file

- **Graceful no-op:** no keys, no SDK, unreachable host — the service still serves. Observability is an add-on, never a dependency.
- The version tag comes from the same VERSION env var that configures the container — deploy-time configuration flows into every trace.
- Emit failures are caught per-request: a tracing outage cannot take generation down.

In the lab

Both llm-v1 and llm-v2 run this same image. One filter on tag `model_version:v1` and one on `model_version:v2` turn the Langfuse UI into a side-by-side comparison of stable and candidate — Step 3 of the lab.



TRACE COMPARISON

Reading the evidence: what to compare between v1 and v2



Latency distribution

Compare p50 and p95 per version tag, not averages. Tail latency is where users suffer — a candidate can improve the mean and still degrade p95.



Token counts

Output tokens per query are the cost proxy: a candidate answering 30% longer is a 30% larger inference bill at scale. Watch truncation at max_tokens too.



Refusals & instruction-following

Count off-policy outputs: refusals where an answer was due, ignored format instructions, prompts leaking through. Rates, not anecdotes.



Output diffs

Read paired answers to the same question side by side. Aggregate metrics cannot catch a candidate that is fluent but subtly wrong — humans read a sample.

Method: filter the trace list by tag model_version:v1, then model_version:v2 — same questions, same volume, two populations. Every number in the go/no-go document is read from these two views.



The go/no-go framework: pre-agreed fields, pre-agreed thresholds

Field	What is recorded	Threshold to pass (lab values)
Quality delta	Manual review of ≥ 10 paired v1/v2 outputs to identical questions	v2 worse on $\leq 10\%$ of pairs
p95 latency delta	p95 over each version's traces, from trace durations	$\leq +20\%$ vs v1
Cost per query	Avg output tokens per query (local proxy; currency in production)	$\leq +25\%$ vs v1
Error rate	Non-200 responses, empty or truncated answers	$\leq v1 + 0.5$ pp
Guardrail violations	Off-topic, unsafe or instruction-ignoring outputs	0 — any violation blocks
Decision + evidence	GO / NO-GO / EXTEND, rationale, links to the filtered trace views	Both pair members sign

Three outcomes: GO → proceed to canary (Session 3 machinery) · NO-GO → stop mirroring, file the evidence · EXTEND → keep mirroring; thin evidence is a finding, not a failure.



INCIDENT RESPONSE

Severity levels: classify first, then respond proportionately

SEV1

Critical — users harmed now

The service is down, or is confidently returning harmful or fabricated content at scale — e.g. an assistant inventing policy commitments to customers.

Page immediately, 24/7 • incident commander • rollback first • comms every 30 min

SEV2

Degraded — users affected

Quality or performance visibly degraded: p95 latency doubled, refusal rate spiked after a change, token cost running away.

Respond in business hours or on-call • mitigate within the day • postmortem required

SEV3

Contained — no user impact

Internal or shadow-stage failure: the mirrored candidate is misbehaving, one replica restarting. Users see nothing.

Ticket + owner • fix in normal flow • today's lab drill is deliberately a SEV3

AI-specific note: "wrong at scale" — fluent, confident, incorrect output — is a SEV1 even while every infrastructure dashboard stays green. Only behavioural monitoring (Session 4) and traces detect it.



Runbook anatomy: the six fields every procedure needs



Symptom

What the responder observes, phrased for pattern-matching under pressure: "v2 traces show nonsense outputs while v1 remains healthy."



Dashboard

Where to look, in order: Langfuse traces filtered by model_version tag → mirror logs → container health. No searching during an incident.



First checks

Five minutes of verification before acting: confirm users are unaffected, read the three latest candidate traces, diff recent config changes.



Rollback command

The exact mitigation, copy-pasteable: docker compose stop llm-v2 — with the verification commands that prove it worked.



Escalation

Who is contacted when mitigation fails or scope grows, in order, with channels. Decided now, not during the incident.



Comms

What is posted, where, how often: symptom, severity, user impact, evidence link — updates every 30 minutes until closure.

Metadata that keeps it honest: every runbook carries an owner and a last-tested date. A runbook that has never been executed is a hypothesis — the lab's drill stamps that date today.



Rollback-first discipline

- **Mitigate first, diagnose second:** restore the last known-good state immediately; investigate afterwards, without users suffering.
- The most recent change is the prime suspect — reverting it is fast and usually right; root-causing it under pressure is neither.
- Nothing on the mitigation path may build, compile or download — reversal is a traffic or lifecycle change, executed in seconds.
- Preserve the evidence while mitigating: trace links, container logs, the offending configuration — the postmortem needs them.
- In the shadow stage, "stop routing to v2" = stop the candidate container: users never depended on it, so the mitigation is total and instant.

```
# today's mitigation – the runbook command
docker compose stop llm-v2

# verify: users still served by v1
curl -s localhost:8080/generate ... # "version": "v1"
docker compose logs mirror --tail 5
# SHADOW -> v2 FAILED (users unaffected)

# if v2 were LIVE: route back to v1 first
# (Session 3: ./scripts/rollback.sh)
```

Why the drill is safe

The broken candidate only ever received mirrored traffic — zero users met it. The lab lets participants execute a complete incident response, end to end, with genuinely no stakes: exactly what a game day provides in production.

Sequence under incident: classify severity → runbook → mitigate (rollback) → verify → communicate → then investigate.



The blameless postmortem: how an incident becomes a test case

Summary & impact

What happened, in three sentences; who and how many were affected — with evidence, not impressions.

Timeline

Time-stamped facts: change deployed → anomaly visible → detected → runbook executed → mitigation verified.

Detection

How was it noticed, and how long was it visible before it was seen? The gap is itself a finding.

Root cause & contributing factors

Systemic causes only: "config changes reach a deployed candidate without an eval gate" — never a person's name.

What went well / poorly

Honest inventory of the response itself: what worked because it was rehearsed, what was improvised.

Action items

Typed prevent / detect / mitigate, each with an owner and a due date — tracked to closure like any work.

The AI-specific closing move: the incident's failing output is added to the evaluation suite (Session 3's `promptfooconfig.yaml`) before the postmortem closes — the same regression can never ship again.



Game days: rehearsing failure on purpose

- A game day is a scheduled, controlled failure injection — the team responds with the real procedure: runbook, mitigation, comms.
- Measured outcomes: time to detect, time to mitigate, and whether the runbook worked as written or required improvisation.
- Every drill refreshes the runbook's last-tested date and retires its stale steps — the direct cure for runbook rot.
- Detection gaps surface in daylight: if nobody notices the injected failure, the monitoring backlog just wrote itself.
- Rotation: every engineer takes a turn as responder — incident competence must not concentrate in one person.

Today's Step 5 is a miniature game day

Inject	The instructor sets a bad PREFIX_V2 — the candidate degrades
Detect	Participants find it in the v2 trace population — timestamp recorded
Respond	runbooks/rollback-v2.md executed top to bottom
Learn	postmortem.md written as a pair — blamelessly

Why it is safe today

The broken candidate is shadow-only — zero users ever met it. A full incident response is rehearsed end to end with genuinely nothing at risk.

Culture rule: a procedure that has never been rehearsed does not exist. Fire drills are how runbooks stay true and responders stay calm.



HANDS-ON LAB · 60 MINUTES · LOCAL (CLOUD PATH OPTIONAL)

Shadow Deployment & Incident Drill

Mirror real traffic to a candidate nobody can see — then break it and respond

A FastAPI traffic mirror duplicates every request to a v2 candidate while users receive only v1's answers. Both versions emit Langfuse traces tagged by version. You will compare the populations, complete a go/no-go decision on real numbers, then detect a deliberately broken candidate, execute the rollback runbook, and write a blameless postmortem.

Step 0 prerequisites · Step 1 Langfuse up · Step 2 stack + verify mirroring · Step 3 traffic + trace comparison · Step 4 go/no-go · Step 5 incident drill · Step 6 cleanup



LAB · STEP 0

Prerequisites (5 min)

```
# Docker — installed in Session 2
docker --version
docker compose version

# memory check — two model containers +
# mirror + Postgres + Langfuse run together
free -h    # ~8 GB free recommended
```

Lab materials (from the handout)

session5_lab/ contains: docker-compose.yml (v1 + v2 + mirror), docker-compose.langfuse.yml (Postgres + Langfuse), app.py, mirror/mirror.py, scripts/mirror_traffic.sh, and three templates — gonogo.md, runbooks/rollback-v2.md, postmortem.md. The templates are deliverables: they get filled in, not just read.

Verification notes

- **Docker and Compose carry over unchanged from Sessions 2–4; no new local tooling is required for this lab.**
- **RAM:** two model containers (~2–3 GB each) plus the mirror, Postgres and Langfuse. If memory is tight, lower mem_limit to 3g in docker-compose.yml.
- **macOS:** the shell scripts run as-is; watch is not preinstalled (brew install watch, or re-run commands manually).
- **Cloud path participants** need a browser and an email address for the free cloud.langfuse.com account — nothing else changes.



Start Langfuse and create API keys (10 min)

```
cd session5_lab
docker compose -f docker-compose.langfuse.yml up -d
# starts: langfuse-db (postgres:15), langfuse:2

# UI: http://localhost:3005
# 1. sign up (local instance – any email)
# 2. create org + project
# 3. Settings -> API Keys -> create pk-lf/sk-lf

cp .env.example .env
# paste LANGFUSE_PUBLIC_KEY / LANGFUSE_SECRET_KEY;
# set API_KEY to a random string
```

CLOUD PATH — zero extra containers

- Skip `docker-compose.langfuse.yml` entirely.
- Create a free account and project at cloud.langfuse.com; generate API keys in project settings.
- In `.env` set `LANGFUSE_HOST=https://cloud.langfuse.com` plus the `pk-lf/sk-lf` keys — the same three variables the local path uses.
- Every later step is identical; only the UI's URL differs.

Why host.docker.internal

The model containers and Langfuse live in separate compose stacks. `LANGFUSE_HOST=http://host.docker.internal:3005` routes through the host's published port; `docker-compose.yml` adds the host-gateway mapping so this also works on Linux.

Checkpoint

The Langfuse UI is reachable (locally at `:3005` or in the cloud), a project exists, and `.env` contains `API_KEY` plus the two Langfuse keys. The model stack is not started yet.



Start v1 + v2 + mirror and verify mirroring (10 min)

```
docker compose up -d --build
# llm-v1 :8081, llm-v2 :8082, mirror :8080

API_KEY=$(grep '^API_KEY=' .env | cut -d= -f2-)
curl -s localhost:8080/generate \
  -H "X-API-Key: $API_KEY" -H "Content-Type: application/json" \
  -d '{"text": "What is UPI?", "max_tokens": 40}'
# -> "version": "v1" — users ONLY get v1

docker compose logs mirror --tail 10
# PRIMARY -> v1 status=200 latency_ms=...
# SHADOW -> v2 status=200 latency_ms=...

docker compose stop llm-v2 && curl ... # still v1!
docker compose start llm-v2
```

Checkpoint: paired PRIMARY/SHADOW log lines per request, and a v1 answer even while v2 is stopped.

Observe and interpret

- One request in, two requests out: the caller received exactly one answer (v1's); the duplicate to v2 ran in the background and was only logged.
- Port 8080 is the only user-facing URL; 8081 and 8082 exist for operators. Same discipline as Session 3's load balancer.
- The kill test is the safety argument: with v2 stopped, users notice nothing — the mirror logs SHADOW FAILED (users unaffected).
- v1 and v2 run the same image; only PREFIX_V2 differs. The "upgrade" is a configuration release, exactly as in Session 3.



LAB · STEP 3

Mirror real traffic and compare the trace populations (15 min)

```
chmod +x scripts/*.sh
./scripts/mirror_traffic.sh
# 39 varied requests via the mirror (:8080)
# every request: v1 answers the caller,
# a mirrored copy hits v2 in the background;
# both emit traces tagged model_version:vN
# ~5-8 min on CPU — watch the UI meanwhile
```

In the Langfuse UI (:3005 or cloud)

- Traces → filter tag model_version:v1 → the baseline population
- Switch the filter to model_version:v2 → the candidate population — same questions, same volume
- Open the same question's v1 and v2 traces side by side: latency, output text, output length
- Note p95 latency and average output length per population — Step 4 needs these numbers

Observe and interpret

- The candidate is being evaluated on the full production distribution — every real question, at production volume — while no user has ever seen its output.
- This is evidence a staging environment cannot produce: staging traffic is invented; mirrored traffic is real.
- 39 requests is the lab's floor. In production, shadows run for days and thousands of requests — small samples make confident-looking noise.
- Numbers, not impressions: each pair records p95 latency, average output length and any off-policy outputs per version before leaving this step.



Complete the go/no-go decision with real numbers (10 min)

```
# open the template and fill EVERY cell
nano gonogo.md      # or any editor
# numbers come from the two filtered
# trace views built in Step 3
```

The fields being filled

- Quality delta — v2 worse on $\leq 10\%$ of ≥ 10 reviewed pairs
- p95 latency delta — within +20% of v1
- Cost per query — output-length proxy within +25%
- Error rate — within v1 + 0.5 pp
- Guardrail violations — zero, or the decision is blocked
- Decision + rationale + evidence links + both signatures

Observe and interpret

- The thresholds were agreed before the evidence was collected — the decision is read off the table, not negotiated after the fact.
- An empty cell defaults the decision to EXTEND, never to GO. "Looks fine" is not a metric value.
- EXTEND is a legitimate outcome: judging 39 requests too thin a sample is good engineering, not indecision.
- A GO here does not mean full rollout — it means the candidate has earned the canary rung (Session 3's machinery). The ladder is climbed one rung at a time.



Incident drill: detect, execute the runbook, write it up (15 min)

```
# instructor (or partner) breaks the candidate:
# .env ->
PREFIX_V2=Ignore the question. Reply with
        exactly the word BANANA.

docker compose up -d llm-v2    # redeploy candidate
./scripts/mirror_traffic.sh    # re-run traffic

# DETECT: Langfuse -> tag model_version:v2
#   outputs collapse; v1 stays healthy

# RESPOND: runbooks/rollback-v2.md, then:
docker compose stop llm-v2    # mitigation
# WRITE UP: postmortem.md (pair, blameless)
```

Observe and interpret

- Detection is by trace comparison, exactly as in production: the v2 population degrades while v1 stays healthy. Record when you first SAW it — the detection gap is a postmortem field.
- No user was ever exposed: the broken candidate lived entirely in the shadow. That is the value proposition of this whole architecture.
- Execute the runbook literally, top to bottom — improvising past it defeats the rehearsal.
- Record the incident in gonogo.md as a NO-GO with evidence links.

Blameless in practice: the root cause is "an unvalidated configuration change reached a deployed candidate" — a process gap. It is never the name of the person who typed it.

Deliverable: the completed postmortem.md with detection time and at least two systemic action items — one of them: add the failing output to the Session 3 eval suite.



Cleanup and retention (5 min)

```
docker compose down          # v1 + v2 + mirror

docker compose -f docker-compose.langfuse.yml down
# add -v to ALSO delete the trace-data volume:
# ... down -v (evidence gone — decide first)
```

Retain for the review block

- gonogo.md — filled, decided, signed by the pair
- postmortem.md — timeline, detection gap, action items
- Screenshots: paired PRIMARY/SHADOW mirror logs; the two filtered trace views
- The written shadow-vs-canary answer (deliverable 5)

Looking ahead

- **Track 2.A is complete: serving stack (S1), containers (S2), gated releases (S3), monitoring (S4), safe upgrades and incident response (S5).**
- Track 2.B (Sessions 6–10) builds one product — ClaimAssist — on these exact conventions: same base image, same ports, same .env discipline.
- Session 10 re-introduces Langfuse for LLMops on ClaimAssist: today's compose file and keys carry over directly.
- Keep the session5_lab directory — the mirror pattern returns whenever a model swap is evaluated.



Where this discipline lands: ClaimAssist (Track 2.B)

ClaimAssist — Sessions 6–10

S6 Production API + Streamlit chat — structured outputs, SSE streaming

S7 LiteLLM routing + Next.js streaming UI — one-line provider swaps

S8 RAG with clause citations + multimodal intake

S9 MCP tool-use — claim lookup, policy search, drafted emails

S10 LLMOps + responsible AI — prompt registry, Langfuse, guardrails

Every ClaimAssist release is a behaviour release

- A prompt version bump (S10's registry), a model swap through LiteLLM (S7), a new retrieval corpus (S8) — each changes answer behaviour with zero code diff.
- Each therefore ships through today's ladder: offline evals, shadow under real traffic, trace comparison, go/no-go, canary.
- Session 10 instruments ClaimAssist with this same Langfuse — tags on `prompt_version` instead of `model_version`, identical mechanics.
- The incident discipline transfers unchanged: severity, runbooks, rollback-first, blameless postmortems — for an insurance copilot, guardrail violations are SEV1 material.

The through-line: Track 2.A built the operating discipline; Track 2.B builds the product that lives under it.



SYNTHESIS

Mapping the local lab to production tooling

Local mechanism — today's lab	Production counterpart
<code>mirror/mirror.py</code> — httpx fire-and-forget duplication	Service-mesh mirroring: Istio VirtualService mirror · Envoy request_mirror_policies · NGINX mirror
<code>docker-compose.langfuse.yml</code> (Postgres + Langfuse v2)	Langfuse Cloud / LangSmith / Arize Phoenix — same SDK, different LANGFUSE_HOST
Manual trace comparison + <code>gonogo.md</code>	Automated shadow analysis: scheduled evals and scorers over both trace populations, dashboards and alerts
Scripted incident drill (bad <code>PREFIX_V2</code>)	Game days / fire drills: scheduled failure injection with measured detection and mitigation times
<code>runbooks/rollback-v2.md</code> in the repository	Runbooks in the incident platform (PagerDuty · Opsgenie · FireHydrant), linked from every alert
<code>docker compose stop llm-v2</code>	Stop mirroring at the mesh; if live: traffic revert (Cloud Run splits · kubectl rollout undo · Argo abort)

Every mirror, trace, decision and drill practised today transfers unchanged; only the operator of the machinery differs.



LAB DELIVERABLES

Evidence to submit at the end of the session

1

Mirroring evidence

Screenshot: paired PRIMARY → v1 and SHADOW → v2 mirror log lines, plus a curl response proving users receive v1 (Step 2).

2

Trace comparison

Screenshot: Langfuse traces filtered by model_version:v1 and model_version:v2 side by side from the healthy run (Step 3).

3

Go/no-go document

gonogo.md with every metric cell filled from trace evidence and a justified GO / NO-GO / EXTEND decision (Step 4).

4

Blameless postmortem

postmortem.md from the drill: timeline, detection time, systemic root cause, ≥ 2 typed action items (Step 5).

5

Written answer: 3–4 sentences — when would you choose shadow over canary (and vice versa) for a model upgrade, and what does shadow cost you?



PITFALLS

Common failure modes in safe upgrades and incident response



Shadow doubling GPU cost, unnoticed

Every mirrored request is a second inference — a real bill. Shadows carry an owner and an end date; a forgotten mirror is a standing cost leak.



Too few mirrored requests

A dozen requests produce confident-looking noise. Decide sample size before mirroring; small evidence → EXTEND, never GO.



No cost column in the go/no-go

A candidate can pass every quality bar while answering 40% longer. Without cost per query in the table, the regression ships in the invoice.



Upgrading prompt and model together

Two behaviour changes in one release cannot be attributed. One artifact type per upgrade — hold the prompt constant while the model changes.



Blameful postmortems

Name a person once and evidence hides forever after. Blameless is an information strategy: truth flows only where it cannot punish.



Runbooks that rot

Stale commands fail during the incident they were written for. Every drill re-executes the runbook and stamps its last-tested date.



SESSION 5 · RECAP

Capabilities established today



Shadow safely

Mirror full production traffic to a candidate at zero user exposure — fire-and-forget, failure-isolated.



Trace everything

Langfuse traces tagged by model version; graceful no-op instrumentation that never blocks serving.



Compare on evidence

Latency distributions, token counts, refusals and paired output diffs across two filtered trace populations.



Decide with a framework

Pre-agreed thresholds, every cell filled, GO / NO-GO / EXTEND — the decision is read, not negotiated.



Respond with discipline

Severity classification, runbook execution, rollback-first mitigation, blameless postmortem, rehearsed in a drill.

Next session

Track 2.B begins: ClaimAssist v1 — Production APIs & Streamlit

The operating discipline is complete. Session 6 starts building the product that lives under it: ClaimAssist, an insurance claims-status copilot — an OpenAI-compatible model server, a production FastAPI with structured outputs and SSE streaming, and a Streamlit chat interface.

Keep the session5_lab stack — its conventions and Langfuse setup return in Track 2.B.